

C++ STL Notes

Everything from the Standard Template Library you need for online tests and coding interviews, explained simply, with code you can trust. Every snippet was compiled and tested with g++ (C++17), and every worked example shows the **real output** from running it.

- | | | | |
|-----------|----------------------------------|-----------|------------------------------|
| 1 | Starter template & fast input | 2 | Data types & limits |
| 3 | Reading the constraints | 4 | vector |
| 5 | pair, tuple, structured bindings | 6 | string |
| 7 | stack, queue, deque | 8 | priority_queue (heap) |
| 9 | set & multiset | 10 | map, unordered_map |
| 11 | Algorithms library | 12 | Bits, bitset & maths helpers |
| 13 | Which container should I use? | 14 | Time complexity cheat table |
| 15 | Ready-to-use patterns | 16 | Mistakes that cost marks |
| 17 | STL interview questions | 18 | Before-you-submit checklist |

How to use these notes: read one section a day, then type the code yourself from memory without looking. The goal is that in a test you never have to stop and remember syntax.

1 Starter template & fast input

Type this at the start of every practice problem until you can write it without thinking. On HackerRank function-stub problems, you only fill in the function, so you won't need the `main()` part there.

```
#include <bits/stdc++.h>          // includes the whole standard library (works on g++)
using namespace std;
using ll = long long;           // write ll instead of long long
const int MOD = 1e9 + 7;       // the usual modulus in counting problems

int main() {
    ios::sync_with_stdio(false); // makes cin/cout much faster
    cin.tie(nullptr);

    int t; cin >> t;             // number of test cases
    while (t--) {
        int n; cin >> n;
        vector<int> a(n);
        for (auto &x : a) cin >> x; // read the array
        // ... solve ...
        cout << "answer" << "\n"; // use "\n", not endl (endl is slow)
    }
}
```

Why the two lines in main? By default C++ keeps `cin` in sync with old C input, which is slow. Turning that off can make input 5–10× faster. Once you use them, **don't mix** `cin` with `scanf/printf`.

Reading tricky input

```
string line; getline(cin, line); // read a whole line with spaces
cin.ignore();                   // skip the leftover '\n' before getline
while (cin >> x) { ... }       // read until input ends
```

Worked examples

Every example in these notes is a complete program. The first two lines (`#include <bits/stdc++.h>` and `using namespace std;`) are left out to save paper. The output shown is the real output from running the program on that input.

EXAMPLE Many test cases

The first line has T, the number of test cases. Each test case has n, then n numbers. Print the sum of each test case.

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t; cin >> t;
    while (t--) {
        int n; cin >> n;
        long long sum = 0;           // long long: the sum can be big
        for (int i = 0; i < n; i++) {
            long long x; cin >> x;
            sum += x;
        }
        cout << sum << "\n";
    }
}
```

INPUT

```
3
3
1 2 3
1
1000000000
2
1000000000 1000000000
```

OUTPUT

```
6
1000000000
2000000000
```

How it works: the while(t--) loop runs exactly T times. The third answer (2×10^9) would not fit in an int, which is why the sum is a long long.

2 Data types & limits

Type	Range (about)	Use it for
int	$\pm 2.1 \times 10^9$	normal counts and indices
long long	$\pm 9.2 \times 10^{18}$	sums, products, anything that can pass 2×10^9
unsigned long long	0 to 1.8×10^{19}	hashing, bit tricks
double	about 15 correct digits	geometry, probability
long double	about 18 correct digits	when double isn't precise enough
char	-128 to 127	letters; c - 'a' gives 0–25

```
INT_MAX, INT_MIN, LLONG_MAX, LLONG_MIN // built-in limits
ll x = 1LL * a * b; // multiply two ints safely: 1LL makes it long long FIRST
ll big = 1e18; // fine for initial "infinity" values
cout << fixed << setprecision(6) << ans; // print a double with 6 decimals
```

Overflow is the #1 reason hidden test cases fail. `int a = 100000, b = 100000; ll c = a * b;` is still wrong, because `a * b` is calculated as `int` first. Write `1LL * a * b`.

Worked examples

EXAMPLE Seeing the limits

Print the biggest `int`, the biggest long long, and 100000×100000 calculated safely.

```
int main() {
    int a = 100000, b = 100000;
    cout << INT_MAX << "\n";
    cout << LLONG_MAX << "\n";
    cout << 1LL * a * b << "\n";          // 1LL turns the multiplication into long long
    double pi = acos(-1.0);
    cout << fixed << setprecision(4) << pi << "\n";
}
```

OUTPUT

```
2147483647
9223372036854775807
100000000000
3.1416
```

How it works: $100000 \times 100000 = 10^{10}$, which is bigger than `INT_MAX` (about 2.1×10^9). Writing `a * b` would overflow; `1LL * a * b` is correct.

3 Reading the constraints

A computer does roughly 10^8 simple operations per second. So the limit on n tells you how fast your solution has to be, and often hints at the idea the setter wants.

If n is up to	Aim for	Usually means
10–12	$O(n!)$	try all permutations (<code>next_permutation</code>)
20–25	$O(2^n)$	bitmask, backtracking, meet in the middle
100	$O(n^4)$	small DP
500	$O(n^3)$	interval DP, Floyd–Warshall
5 000	$O(n^2)$	2D DP, double loop
$10^5 - 10^6$	$O(n \log n)$ or $O(n)$	sorting, heap, set/map, binary search, two pointers
$10^9 - 10^{18}$	$O(\log n)$ or $O(1)$	binary search on answer, maths, fast power

4 vector

A **vector** is an array that can grow. It's the container you'll use most. Elements sit next to each other in memory, so accessing `v[i]` is instant.

```

vector<int> v;                // empty
vector<int> v(n);             // n zeros
vector<int> v(n, -1);         // n copies of -1
vector<int> v = {4, 1, 3};    // with values

v.push_back(7);              // add at the end          0(1) on average
v.pop_back();                // remove last              0(1)
v.back();                    // last element
v.front();                   // first element
v.size();                    // number of elements (unsigned! see the warning below)
v.empty();                   // true if size is 0
v.clear();                   // remove everything
v.resize(10);                // change size (new slots become 0)
v.assign(n, 0);              // reset to n zeros
v.insert(v.begin() + i, x);   // insert at index i      0(n), slow
v.erase(v.begin() + i);      // remove index i         0(n), slow

for (int x : v) ...           // loop by value (a copy)
for (int &x : v) x *= 2;       // loop by reference (can change elements)

```

2D vectors (grids, DP tables, graphs)

```

vector<vector<int>> grid(n, vector<int>(m, 0)); // n rows, m columns, all 0
grid[i][j] = 5;

vector<vector<int>> adj(n); // graph adjacency list for n nodes
adj[u].push_back(v);       // edge u -> v

```

v.size() is unsigned. If v is empty, `v.size() - 1` becomes a huge number, not -1. Write `(int)v.size() - 1` or store `int n = v.size();` first.

Pass vectors by reference into functions: `void dfs(int u, vector<int>& vis)`. Without the `&`, C++ copies the whole vector every call, which causes TLE in recursion.

Worked examples

EXAMPLE Basic vector operations

Read n numbers. Add 10 at the end, then print the size, first and last elements, the sum, and the vector reversed.

```
int main() {
    int n; cin >> n;
    vector<int> v(n);
    for (auto &x : v) cin >> x;
    v.push_back(10);
    cout << "size = " << v.size() << "\n";
    cout << "first = " << v.front() << ", last = " << v.back() << "\n";
    long long sum = 0;
    for (int x : v) sum += x;
    cout << "sum = " << sum << "\n";
    reverse(v.begin(), v.end());
    cout << "reversed:";
    for (int x : v) cout << " " << x;
    cout << "\n";
}
```

INPUT

```
5
3 1 4 1 5
```

OUTPUT

```
size = 6
first = 3, last = 10
sum = 24
reversed: 10 5 1 4 1 3
```

EXAMPLE Rotate an array to the right by k

Shift every element k places to the right; elements that fall off the end come back at the front.

```
int main() {
    int n, k; cin >> n >> k;
    vector<int> v(n);
    for (auto &x : v) cin >> x;
    k %= n;                                // rotating by n changes nothing
    vector<int> res(n);
    for (int i = 0; i < n; i++)
        res[(i + k) % n] = v[i];          // element i moves to i + k (wrapping round)
    for (int x : res) cout << x << " ";
    cout << "\n";
}
```

INPUT

```
7 3
1 2 3 4 5 6 7
```

OUTPUT

```
5 6 7 1 2 3 4
```

How it works: $(i + k) \% n$ wraps indices past the end back to the start.

Time: $O(n)$

EXAMPLE Move all zeros to the end

Keep the order of the non-zero numbers and push every 0 to the end, in place.

```
int main() {
    int n; cin >> n;
    vector<int> v(n);
    for (auto &x : v) cin >> x;
    int pos = 0; // next place for a non-zero number
    for (int i = 0; i < n; i++)
        if (v[i] != 0) swap(v[pos++], v[i]);
    for (int x : v) cout << x << " ";
    cout << "\n";
}
```

INPUT

```
6
0 1 0 3 12 0
```

OUTPUT

```
1 3 12 0 0 0
```

How it works: `pos` marks where the next non-zero belongs. Every non-zero is swapped into place, so the zeros drift to the back.

Time: $O(n)$, no extra memory

EXAMPLE 2D vector: row and column sums

Read an $r \times c$ matrix and print the sum of each row and each column.

```
int main() {
    int r, c; cin >> r >> c;
    vector<vector<int>> g(r, vector<int>(c));
    for (auto &row : g)
        for (auto &x : row) cin >> x;
    vector<long long> rowSum(r, 0), colSum(c, 0);
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++) {
            rowSum[i] += g[i][j];
            colSum[j] += g[i][j];
        }
    cout << "rows:";
    for (auto s : rowSum) cout << " " << s;
    cout << "\ncols:";
    for (auto s : colSum) cout << " " << s;
    cout << "\n";
}
```

INPUT

```
2 3
1 2 3
4 5 6
```

OUTPUT

```
rows: 6 15
cols: 5 7 9
```

5 pair, tuple, structured bindings

```
pair<int, int> p = {3, 5};
p.first; p.second;           // 3, 5
auto p2 = make_pair(1, "x");

vector<pair<int,int>> v;       // e.g. (value, index) or (start, end)
sort(v.begin(), v.end());    // sorts by first, then by second

tuple<int,int,int> t = {1, 2, 3};
get<0>(t);                   // 1

auto [a, b] = p;             // structured binding (C++17): a = 3, b = 5
for (auto &[key, val] : mp) ... // very handy with maps
```

Pairs compare **first by .first, then by .second**. That's why sorting a `vector<pair<int,int>>` "just works", and why `{distance, node}` pairs work nicely in a priority queue.

Worked examples

EXAMPLE Sort students: marks high to low, then name A to Z

Sort by marks in descending order. If two students have the same marks, put them in alphabetical order.

```
int main() {
    int n; cin >> n;
    vector<pair<string,int>> s(n);
    for (auto &[name, marks] : s) cin >> name >> marks;
    sort(s.begin(), s.end(), [](const pair<string,int>& a, const pair<string,int>& b) {
        if (a.second != b.second) return a.second > b.second; // more marks first
        return a.first < b.first; // then by name
    });
    for (auto &[name, marks] : s) cout << name << " " << marks << "\n";
}
```

INPUT

```
5
riya 85
aman 92
kabir 85
zoya 70
dev 92
```

OUTPUT

```
aman 92
dev 92
kabir 85
riya 85
zoya 70
```

How it works: the lambda returns true when *a* should come before *b*. It compares marks first and only looks at names when the marks are equal.

EXAMPLE Maximum meetings in one room

Each meeting has a start and end time. What is the largest number of meetings that don't overlap? (A meeting can start only after the previous one ends.)

```
int main() {
    int n; cin >> n;
    vector<pair<int,int>> m(n);           // stored as {end, start}
    for (auto &[e, s] : m) cin >> s >> e;
    sort(m.begin(), m.end());           // pairs sort by .first = end time
    int count = 0, lastEnd = INT_MIN;
    for (auto &[e, s] : m)
        if (s > lastEnd) { count++; lastEnd = e; }
    cout << count << "\n";
}
```

INPUT

```
6
1 2
3 4
0 6
5 7
8 9
5 9
```

OUTPUT

```
4
```

How it works: store the end time first, so a plain `sort` orders meetings by when they finish. Always taking the meeting that ends earliest leaves the most room for the rest.

Time: $O(n \log n)$

EXAMPLE tuple: sort points by distance from the origin

Sort points by their distance from (0, 0), closest first.

```
int main() {
    int n; cin >> n;
    vector<tuple<int,int,int>> p;         // {squared distance, x, y}
    for (int i = 0; i < n; i++) {
        int x, y; cin >> x >> y;
        p.push_back({x * x + y * y, x, y});
    }
    sort(p.begin(), p.end());           // compares the first item, then the next...
    for (auto &[d, x, y] : p) cout << x << " " << y << "\n";
}
```

INPUT

```
4
3 4
1 1
-2 0
0 1
```

OUTPUT

```
0 1
1 1
-2 0
3 4
```

How it works: compare squared distances ($x^2 + y^2$) to avoid square roots and decimal errors. Putting the distance first in the tuple makes the default sort do the work.

6 string

```
string s = "hello";
s.size(); s.length();           // 5
s[0];                           // 'h'
s += " world";                  // append (fast)
s.push_back('!'); s.pop_back();
s.substr(pos, len);             // part of the string, e.g. s.substr(6, 5) = "world"
s.find("lo");                   // first index of "lo", or string::npos if not found
if (s.find("zz") == string::npos) { /* not found */ }
s.insert(2, "XY"); s.erase(2, 2);
reverse(s.begin(), s.end());
sort(s.begin(), s.end());       // sort the letters (useful for anagrams)

to_string(42);                  // "42"
stoi("42");                     // 42      (string -> int)
stoll("10000000000");           // string -> long long

// character helpers
isdigit(c); isalpha(c); islower(c); isupper(c);
tolower(c); toupper(c);
int idx = c - 'a';               // 'a' -> 0, 'b' -> 1 ...
char ch = 'a' + idx;

// split a line into words
stringstream ss(line); string word;
while (ss >> word) { ... }
```

`s = s + 'x';` inside a loop is $O(n)$ each time because it builds a new string. Use `s += 'x';` or `s.push_back('x');` instead.

Worked examples

EXAMPLE Palindrome check, ignoring case and symbols

For each line, print YES if it reads the same forwards and backwards when you keep only letters and digits and ignore case.

```
int main() {
    string line;
    while (getline(cin, line)) {
        string s;
        for (char ch : line)
            if (isalnum(ch)) s += tolower(ch);
        string r = s;
        reverse(r.begin(), r.end());
        cout << (s == r ? "YES" : "NO") << "\n";
    }
}
```

INPUT

```
A man, a plan, a canal: Panama
race a car
```

OUTPUT

```
YES
NO
```

How it works: `getline` reads the whole line including spaces. Then we build a clean lowercase string and compare it with its reverse.

EXAMPLE Count words and reverse their order

Words can be separated by several spaces. Print the number of words, then the words in reverse order.

```
int main() {
    string line; getline(cin, line);
    stringstream ss(line);
    vector<string> words;
    string w;
    while (ss >> w) words.push_back(w);    // >> skips any amount of spaces
    cout << words.size() << "\n";
    for (int i = (int)words.size() - 1; i >= 0; i--)
        cout << words[i] << (i ? " " : "\n");
}
```

INPUT

```
the sky   is blue
```

OUTPUT

```
4
blue is sky the
```

EXAMPLE Anagram check

Two words are anagrams if they use exactly the same letters. For each pair print YES or NO.

```
int main() {
    int t; cin >> t;
    while (t--) {
        string a, b; cin >> a >> b;
        sort(a.begin(), a.end());
        sort(b.begin(), b.end());
        cout << (a == b ? "YES" : "NO") << "\n";
    }
}
```

INPUT

```
3
listen silent
hello world
rat tar
```

OUTPUT

```
YES
NO
YES
```

How it works: after sorting their letters, anagrams become identical strings. (Faster alternative: compare two count arrays of size 26.)

Time: $O(n \log n)$ per pair

EXAMPLE Letters that repeat

Print every letter that appears more than once in the word, in alphabetical order, with its count.

```
int main() {
    string s; cin >> s;
    int cnt[26] = {0};
    for (char ch : s) cnt[ch - 'a']++; // 'a' -> 0, 'b' -> 1, ...
    for (int i = 0; i < 26; i++)
        if (cnt[i] > 1) cout << char('a' + i) << " " << cnt[i] << "\n";
}
```

INPUT

```
programming
```

OUTPUT

```
g 2
m 2
r 2
```

How it works: for lowercase letters a plain array of 26 counters is simpler and faster than a map.

pop() returns nothing. Read the value first with `top()` / `front()`, then pop. Also never call `top()` or `pop()` on an empty stack or queue, as it crashes the program. Check `!st.empty()` first.

Worked examples

EXAMPLE Balanced brackets (stack)

For each string of brackets, print YES if every bracket is closed in the right order.

```
bool balanced(const string& s) {
    stack<char> st;
    for (char ch : s) {
        if (ch == '(' || ch == '[' || ch == '{') st.push(ch);
        else {
            if (st.empty()) return false;    // closing with nothing open
            char top = st.top(); st.pop();
            if ((ch == ')' && top != '(') || (ch == ']' && top != '[') ||
                (ch == '}' && top != '{')) return false;
        }
    }
    return st.empty();                      // anything left open?
}

int main() {
    int t; cin >> t;
    while (t--) { string s; cin >> s; cout << (balanced(s) ? "YES" : "NO") << "\n"; }
}
```

INPUT

```
3
{[()]}
([]]
((
```

OUTPUT

```
YES
NO
NO
```

How it works: the most recent open bracket must be the first one closed: last in, first out, which is exactly a stack.

Time: $O(n)$

EXAMPLE Evaluate a postfix expression (stack)

Evaluate an expression written in postfix form, e.g. `2 3 +` means $2 + 3$.

```
int main() {
    string tok;
    stack<long long> st;
    while (cin >> tok) {
        if (tok == "+" || tok == "-" || tok == "*" || tok == "/") {
            long long b = st.top(); st.pop();    // right operand is on top
            long long a = st.top(); st.pop();
            if (tok == "+") st.push(a + b);
            else if (tok == "-") st.push(a - b);
            else if (tok == "*") st.push(a * b);
            else st.push(a / b);
        } else st.push(stoll(tok));
    }
    cout << st.top() << "\n";
}
```

INPUT

```
2 3 1 * + 9 -
```

OUTPUT

```
-4
```

How it works: numbers go on the stack; an operator pops two numbers and pushes the result. Here: $3 \cdot 1 = 3$, $2 + 3 = 5$, $5 - 9 = -4$.

EXAMPLE Josephus problem (queue)

n people stand in a circle. Counting starts at person 1, and every k -th person is removed. Print the order people leave, and the survivor.

```
int main() {
    int n, k; cin >> n >> k;
    queue<int> q;
    for (int i = 1; i <= n; i++) q.push(i);
    cout << "removed:";
    while (q.size() > 1) {
        for (int i = 1; i < k; i++) { q.push(q.front()); q.pop(); } // skip k-1 people
        cout << " " << q.front();
        q.pop(); // remove the k-th
    }
    cout << "\nsurvivor: " << q.front() << "\n";
}
```

INPUT

```
7 3
```

OUTPUT

```
removed: 3 6 2 7 5 1
survivor: 4
```

How it works: moving the front person to the back simulates walking round the circle.

Time: $O(n \cdot k)$

EXAMPLE First negative number in every window (deque)

For every window of k consecutive numbers, print the first negative number in it, or 0 if there is none.

```
int main() {
    int n, k; cin >> n >> k;
    vector<int> a(n);
    for (auto &x : a) cin >> x;
    deque<int> dq; // indices of negative numbers in the window
    for (int i = 0; i < n; i++) {
        if (a[i] < 0) dq.push_back(i);
        if (!dq.empty() && dq.front() <= i - k) dq.pop_front(); // left the window
        if (i >= k - 1) cout << (dq.empty() ? 0 : a[dq.front()]) << " ";
    }
    cout << "\n";
}
```

INPUT

```
8 3
12 -1 -7 8 -15 30 16 28
```

OUTPUT

```
-1 -1 -7 -15 -15 0
```

How it works: new negatives join at the back; ones that slide out of the window leave from the front. The front is always the answer.

Time: $O(n)$

8 priority_queue (heap)

A priority queue always gives you the **largest** (or smallest) element quickly. Use it whenever a problem says "top K ", "smallest so far", "always pick the cheapest", or for Dijkstra.

```
priority_queue<int> maxHeap; // biggest on top (default)
priority_queue<int, vector<int>, greater<int>> minHeap; // smallest on top

maxHeap.push(5); //  $O(\log n)$ 
maxHeap.top(); // look at the biggest  $O(1)$ 
maxHeap.pop(); // remove the biggest  $O(\log n)$ 
maxHeap.size(); maxHeap.empty();

// heap of pairs: compares .first, then .second
priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq; // min by .first

// custom rule: smallest .second on top
auto cmp = [](const pair<int,int>& a, const pair<int,int>& b) {
    return a.second > b.second; // note: ">" gives a MIN-heap here
};
priority_queue<pair<int,int>, vector<pair<int,int>>, decltype(cmp)> pq2(cmp);
```

The comparator feels "backwards". In sort, $a < b$ means ascending. In a priority_queue, $a < b$ means the **largest** is on top. So to get the smallest on top you write $>$ (or use `greater`). Trick: *multiply values by -1* to turn a max-heap into a min-heap.

Top K largest: keep a **min-heap of size K**. Push each element; if size goes above K, pop. At the end, the heap holds the K largest. Cost: $O(n \log K)$.

Worked examples

EXAMPLE K-th largest element

Find the k-th largest number without sorting the whole array.

```
int main() {
    int n, k; cin >> n >> k;
    priority_queue<int, vector<int>, greater<int>> pq;    // min-heap
    for (int i = 0; i < n; i++) {
        int x; cin >> x;
        pq.push(x);
        if ((int)pq.size() > k) pq.pop();    // throw away the smallest
    }
    cout << pq.top() << "\n";    // smallest of the k largest
}
```

INPUT

```
6 2
3 2 1 5 6 4
```

OUTPUT

```
5
```

How it works: the heap always holds the k largest numbers seen so far. Its smallest element (the top) is the k-th largest.

Time: $O(n \log k)$

EXAMPLE Connect ropes with minimum cost

Joining two ropes of length a and b costs a + b. Join all ropes into one with the lowest total cost.

```
int main() {
    int n; cin >> n;
    priority_queue<long long, vector<long long>, greater<long long>> pq;
    for (int i = 0; i < n; i++) { long long x; cin >> x; pq.push(x); }
    long long total = 0;
    while (pq.size() > 1) {
        long long a = pq.top(); pq.pop();
        long long b = pq.top(); pq.pop();
        total += a + b;
        pq.push(a + b);    // the joined rope goes back in
    }
    cout << total << "\n";
}
```

INPUT

```
4
4 3 2 6
```

OUTPUT

```
29
```

How it works: always join the two shortest ropes ($2+3 = 5$, then $4+5 = 9$, then $6+9 = 15$; total 29). Long ropes get added fewer times.

Time: $O(n \log n)$

EXAMPLE Merge K sorted lists

Merge several sorted lists into one sorted list.

```
int main() {
    int k; cin >> k;
    vector<vector<int>> lists(k);
    for (auto &l : lists) {
        int m; cin >> m; l.resize(m);
        for (auto &x : l) cin >> x;
    }
    // {value, which list, index inside that list}; smallest value on top
    priority_queue<tuple<int,int,int>, vector<tuple<int,int,int>>, greater<>> pq;
    for (int i = 0; i < k; i++)
        if (!lists[i].empty()) pq.push({lists[i][0], i, 0});
    while (!pq.empty()) {
        auto [val, li, idx] = pq.top(); pq.pop();
        cout << val << " ";
        if (idx + 1 < (int)lists[li].size()) pq.push({lists[li][idx + 1], li, idx + 1});
    }
    cout << "\n";
}
```

INPUT

```
3
3 1 4 7
3 2 5 8
3 3 6 9
```

OUTPUT

```
1 2 3 4 5 6 7 8 9
```

How it works: the heap holds the current front of each list. Take the smallest, then push the next element from the same list.

Time: $O(N \log k)$ for N total numbers

EXAMPLE Top K most frequent numbers

Print the k numbers that appear most often, most frequent first.

```
int main() {
    int n, k; cin >> n >> k;
    unordered_map<int,int> freq;
    for (int i = 0; i < n; i++) { int x; cin >> x; freq[x]++; }
    // min-heap of {count, number}, kept at size k
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;
    for (auto &[num, cnt] : freq) {
        pq.push({cnt, num});
        if ((int)pq.size() > k) pq.pop();
    }
    vector<int> ans;
    while (!pq.empty()) { ans.push_back(pq.top().second); pq.pop(); }
    reverse(ans.begin(), ans.end()); // most frequent first
    for (int x : ans) cout << x << " ";
    cout << "\n";
}
```

INPUT

```
6 2
1 1 1 2 2 3
```

OUTPUT

```
1 2
```

EXAMPLE Hospital queue with a custom comparator

Patients with higher severity are seen first. With equal severity, whoever arrived earlier goes first.

```
struct Patient { string name; int severity, arrival; };
struct Cmp {
    bool operator()(const Patient& a, const Patient& b) const {
        if (a.severity != b.severity) return a.severity < b.severity; // bigger severity on top
        return a.arrival > b.arrival; // earlier arrival on top
    }
};

int main() {
    int n; cin >> n;
    priority_queue<Patient, vector<Patient>, Cmp> pq;
    for (int i = 0; i < n; i++) {
        string name; int sev; cin >> name >> sev;
        pq.push({name, sev, i});
    }
    while (!pq.empty()) { cout << pq.top().name << " "; pq.pop(); }
    cout << "\n";
}
```

INPUT

```
4
ravi 2
sita 5
mohan 5
lata 1
```

OUTPUT

```
sita mohan ravi lata
```

How it works: the comparator returns true when *a* should be **lower** in the heap than *b*. That's why it looks reversed compared with sort.

9 set & multiset

A **set** keeps values **sorted** and **unique**. A **multiset** keeps them sorted but **allows duplicates**. Both are built on a balanced tree, so most operations are $O(\log n)$.

```

set<int> s;
s.insert(4); s.insert(1); s.insert(4); // s = {1, 4} (duplicate ignored)
s.erase(4); // remove by value
s.count(1); // 1 if present, 0 if not
s.find(1) != s.end(); // another way to check
*s.begin(); // smallest
*s.rbegin(); // largest
s.size();

// the superpower: searching for neighbours
auto it = s.lower_bound(x); // first element >= x (or s.end())
auto it2 = s.upper_bound(x); // first element > x
if (it != s.begin()) { auto prev = std::prev(it); } // largest element < x

multiset<int> ms = {2, 2, 3};
ms.erase(ms.find(2)); // removes ONE copy of 2 -> {2, 3}
ms.erase(2); // removes ALL copies of 2 -> {3}

unordered_set<int> us; // not sorted, O(1) average, no lower_bound

```

Two classic bugs. (1) `ms.erase(x)` deletes **every** copy of `x`; use `ms.erase(ms.find(x))` for one. (2) Always use `s.lower_bound(x)`, **not** `lower_bound(s.begin(), s.end(), x)`. The second form is $O(n)$ on a set and causes TLE.

Worked examples

EXAMPLE Distinct values in sorted order

Print how many different numbers there are, and list them in increasing order.

```

int main() {
    int n; cin >> n;
    set<int> s;
    for (int i = 0; i < n; i++) { int x; cin >> x; s.insert(x); }
    cout << s.size() << "\n";
    for (int x : s) cout << x << " ";
    cout << "\n";
}

```

INPUT

```

8
5 3 5 1 3 9 1 5

```

OUTPUT

```

4
1 3 5 9

```

EXAMPLE Floor and ceiling queries

For each query x , print the largest value $\leq x$ (floor) and the smallest value $\geq x$ (ceiling). Print -1 if it doesn't exist.

```
int main() {
    int n, q; cin >> n >> q;
    set<int> s;
    for (int i = 0; i < n; i++) { int x; cin >> x; s.insert(x); }
    while (q--) {
        int x; cin >> x;
        auto it = s.lower_bound(x);           // first value >= x
        int ceil = (it == s.end()) ? -1 : *it;
        auto jt = s.upper_bound(x);           // first value > x
        int floor = (jt == s.begin()) ? -1 : *prev(jt);
        cout << x << ": floor " << floor << ", ceil " << ceil << "\n";
    }
}
```

INPUT

```
4 4
1 4 7 10
5 10 0 11
```

OUTPUT

```
5: floor 4, ceil 7
10: floor 10, ceil 10
0: floor -1, ceil 1
11: floor 10, ceil -1
```

How it works: the element just before `upper_bound(x)` is the largest value $\leq x$. Always check for `begin()` / `end()` before using the iterator.

Time: $O(\log n)$ per query

EXAMPLE Live minimum and maximum with a multiset

Numbers are added (+) and removed (-). After each operation, print the current minimum and maximum.

```
int main() {
    int q; cin >> q;
    multiset<int> ms;
    while (q--) {
        char op; int x; cin >> op >> x;
        if (op == '+') ms.insert(x);
        else ms.erase(ms.find(x));          // remove just ONE copy
        cout << *ms.begin() << " " << *ms.rbegin() << "\n";
    }
}
```

INPUT

```
6
+ 5
+ 1
+ 5
- 5
+ 9
- 1
```

OUTPUT

```
5 5
1 5
1 5
1 5
1 9
5 9
```

How it works: after `- 5` one 5 is still left. With `ms.erase(5)` both would vanish and the answer would be wrong.

Time: $O(\log n)$ per operation

EXAMPLE Longest consecutive sequence (unordered_set)

Find the length of the longest run of consecutive integers (in any order in the array).

```
int main() {
    int n; cin >> n;
    unordered_set<int> s;
    for (int i = 0; i < n; i++) { int x; cin >> x; s.insert(x); }
    int best = 0;
    for (int x : s) {
        if (s.count(x - 1)) continue;          // x is not the start of a run
        int len = 1;
        while (s.count(x + len)) len++;
        best = max(best, len);
    }
    cout << best << "\n";
}
```

INPUT

```
6
100 4 200 1 3 2
```

OUTPUT

```
4
```

How it works: only start counting at numbers whose $x-1$ is missing. Each number is visited at most twice, so the total is $O(n)$ even though there's a loop inside a loop.

Time: $O(n)$ on average

10 map, unordered_map

A map stores **key** → **value** pairs, with each key appearing once. `map` keeps keys sorted (tree, $O(\log n)$); `unordered_map` doesn't (hash table, $O(1)$ on average).

```
map<string, int> mp;
mp["apple"] = 3;           // insert or overwrite
mp["apple"]++;            // counting: missing keys start at 0
mp.count("kiwi");          // 0 or 1    (does NOT create the key)
auto it = mp.find("apple"); // iterator, or mp.end() if missing
if (it != mp.end()) cout << it->first << " " << it->second;
mp.erase("apple");
mp.size();

for (auto &[key, val] : mp) cout << key << " " << val << "\n";    // sorted by key

// map-only (sorted) features
mp.begin()->first;          // smallest key
mp.rbegin()->first;         // largest key
mp.lower_bound(k);          // first key >= k

unordered_map<int, int> freq;
for (int x : a) freq[x]++;  // frequency count in O(n)
freq.reserve(1 << 20);     // optional: fewer rehashes, a bit faster
```

mp[key] creates the key if it's missing. Just reading `if (mp[x] > 0)` adds `x` with value 0. That can change `mp.size()` and break loops. Use `mp.count(x)` or `mp.find(x)` when you only want to check.

When unordered_map is slow (anti-hash tests)

On Codeforces and some OAs, special inputs make many keys land in the same bucket, so `unordered_map` becomes $O(n)$ per operation and you get TLE. Either switch to `map`, or use this well-known custom hash:

```
struct custom_hash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t FIXED = chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + FIXED);
    }
};

unordered_map<long long, int, custom_hash> safeMap;
```


	map / set	unordered_map / unordered_set
Built using	red-black tree (balanced BST)	hash table
Speed	$O(\log n)$ always	$O(1)$ average, $O(n)$ worst
Keys sorted?	yes	no
lower_bound / smallest / largest	yes	no
Key can be a pair or vector?	yes, works directly	needs a custom hash
Pick it when	you need order or neighbours	you only need fast lookups or counts

Worked examples

EXAMPLE Word frequency, most common first

Count each word. Print by count (highest first); break ties alphabetically.

```
int main() {
    map<string,int> cnt;
    string w;
    while (cin >> w) cnt[w]++;
    vector<pair<string,int>> v(cnt.begin(), cnt.end()); // copy map into a vector
    sort(v.begin(), v.end(), [](auto &a, auto &b) {
        if (a.second != b.second) return a.second > b.second;
        return a.first < b.first;
    });
    for (auto &[word, c] : v) cout << word << " " << c << "\n";
}
```

INPUT

```
the cat and the hat and the bat
```

OUTPUT

```
the 3
and 2
bat 1
cat 1
hat 1
```

How it works: you can't sort a map by value, so copy its pairs into a vector and sort that.

EXAMPLE Two Sum

Find two different indices whose values add up to the target.

```
int main() {
    int n, target; cin >> n >> target;
    vector<int> a(n);
    for (auto &x : a) cin >> x;
    unordered_map<int,int> seen;           // value -> index
    for (int i = 0; i < n; i++) {
        int need = target - a[i];
        if (seen.count(need)) { cout << seen[need] << " " << i << "\n"; return 0; }
        seen[a[i]] = i;
    }
    cout << "-1\n";
}
```

INPUT

```
4 9
2 7 11 15
```

OUTPUT

```
0 1
```

How it works: for each number, ask "have I already seen the number that completes it?" A hash map answers that in $O(1)$.

Time: $O(n)$

EXAMPLE Group anagrams

Put words that are anagrams of each other into the same group.

```
int main() {
    int n; cin >> n;
    map<string, vector<string>> groups;
    for (int i = 0; i < n; i++) {
        string w; cin >> w;
        string key = w;
        sort(key.begin(), key.end());           // "eat", "tea", "ate" all become "aet"
        groups[key].push_back(w);
    }
    for (auto &[key, words] : groups) {
        for (auto &w : words) cout << w << " ";
        cout << "\n";
    }
}
```

INPUT

```
6
eat tea tan ate nat bat
```

OUTPUT

```
bat
eat tea ate
tan nat
```

How it works: the sorted word is the same for every anagram, so it works as the map key. A map's value can be a whole vector.

EXAMPLE First character that doesn't repeat

Print the first character that appears exactly once, with its index (or -1).

```
int main() {
    string s;
    while (cin >> s) {
        unordered_map<char,int> cnt;
        for (char ch : s) cnt[ch]++;
        int ans = -1;
        for (int i = 0; i < (int)s.size(); i++)
            if (cnt[s[i]] == 1) { ans = i; break; }
        if (ans == -1) cout << "-1\n";
        else cout << s[ans] << " " << ans << "\n";
    }
}
```

INPUT

```
leetcode
loveleetcode
aabb
```

OUTPUT

```
l 0
v 2
-1
```

How it works: two passes: first count everything, then walk the string in order and stop at the first count of 1.

EXAMPLE Grade bands with map::upper_bound

Marks 0–39 = F, 40–54 = D, 55–69 = C, 70–84 = B, 85+ = A. Print the grade for each score.

```
int main() {
    map<int,char> band = {{0,'F'}, {40,'D'}, {55,'C'}, {70,'B'}, {85,'A'}};
    int score;
    while (cin >> score) {
        auto it = prev(band.upper_bound(score)); // largest key <= score
        cout << score << " -> " << it->second << "\n";
    }
}
```

INPUT

```
39 40 72 100
```

OUTPUT

```
39 -> F
40 -> D
72 -> B
100 -> A
```

How it works: store only where each band *starts*. The band for a score is the largest start $\leq$ score, found with `prev(upper_bound(score))`.

Time: $O(\log n)$ per score

EXAMPLE Counting points: a pair as the map key

Count how many times each (x, y) point appears.

```
int main() {
    int n; cin >> n;
    map<pair<int,int>, int> cnt;           // map works with pair keys directly
    for (int i = 0; i < n; i++) { int x, y; cin >> x >> y; cnt[{x, y}]++; }
    for (auto &[pt, c] : cnt)
        cout << "(" << pt.first << "," << pt.second << ") " << c << "\n";
}
```

INPUT

```
5
1 2
3 4
1 2
0 0
1 2
```

OUTPUT

```
(0,0) 1
(1,2) 3
(3,4) 1
```

How it works: `map` only needs `<` on the key, which pairs already have. An `unordered_map` would need a custom hash.

11 Algorithms library

These work on any range written as `(begin, end)`. For a vector that's `v.begin(), v.end()`; for a plain array `a` of size `n` it's `a, a + n`.

Sorting

```
sort(v.begin(), v.end());           // ascending      O(n log n)
sort(v.begin(), v.end(), greater<int>()); // descending
sort(v.rbegin(), v.rend());         // also descending

// custom rule with a lambda: sort pairs by second value, then first
sort(v.begin(), v.end(), [](const pair<int,int>& a, const pair<int,int>& b) {
    if (a.second != b.second) return a.second < b.second;
    return a.first < b.first;
});
stable_sort(v.begin(), v.end(), cmp); // keeps equal elements in original order
```

Comparator rule: it must return **false when a and b are equal**. Use `<`, never `<=`. Using `<=` can crash `sort` (it breaks the "strict weak ordering" rule).

Binary search (on a SORTED range)

```
// v = {1, 2, 2, 5, 8}
binary_search(v.begin(), v.end(), 5);           // true / false
lower_bound(v.begin(), v.end(), 2) - v.begin(); // 1 -> first index with value >= 2
upper_bound(v.begin(), v.end(), 2) - v.begin(); // 3 -> first index with value > 2
// how many 2s? upper_bound - lower_bound = 3 - 1 = 2
// if nothing is >= x, lower_bound returns v.end() (index = v.size())
```

Everyday helpers

Code	What it does	Time
<code>reverse(v.begin(), v.end())</code>	reverse in place	O(n)
<code>*max_element(v.begin(), v.end())</code>	largest value (drop * to get the iterator)	O(n)
<code>*min_element(v.begin(), v.end())</code>	smallest value	O(n)
<code>accumulate(v.begin(), v.end(), 0LL)</code>	sum (use 0LL so the sum is long long!)	O(n)
<code>count(v.begin(), v.end(), x)</code>	how many times x appears	O(n)
<code>find(v.begin(), v.end(), x)</code>	iterator to first x, or end()	O(n)
<code>fill(v.begin(), v.end(), 0)</code>	set every element to 0	O(n)
<code>iota(v.begin(), v.end(), 0)</code>	fill with 0, 1, 2, 3 ...	O(n)
<code>v.erase(unique(v.begin(), v.end()), v.end())</code>	remove duplicates (sort first!)	O(n)
<code>next_permutation(v.begin(), v.end())</code>	next arrangement; false after the last one	O(n)
<code>nth_element(v.begin(), v.begin()+k, v.end())</code>	puts the k-th smallest at index k	O(n) avg
<code>partial_sum(a.begin(), a.end(), p.begin())</code>	prefix sums	O(n)
<code>rotate(v.begin(), v.begin()+k, v.end())</code>	rotate left by k	O(n)
<code>swap(a, b)</code>	swap two values or containers	O(1)
<code>max({a, b, c})</code>	max of many values	O(k)

```
// all permutations of {1, 2, 3}: start from the SORTED order
sort(p.begin(), p.end());
do { /* use p */ } while (next_permutation(p.begin(), p.end()));
```

Worked examples

EXAMPLE First and last position of x (sorted array)

Print the first and last index of x, or -1 -1 if x is not there.

```
int main() {
    int n, q; cin >> n >> q;
    vector<int> a(n);
    for (auto &x : a) cin >> x;
    while (q--) {
        int x; cin >> x;
        int lo = lower_bound(a.begin(), a.end(), x) - a.begin();
        int hi = upper_bound(a.begin(), a.end(), x) - a.begin();
        if (lo == hi) cout << "-1 -1\n";          // x not present
        else cout << lo << " " << hi - 1 << "\n";
    }
}
```

INPUT

```
6 2
5 7 7 8 8 10
8 6
```

OUTPUT

```
3 4
-1 -1
```

How it works: `lower_bound` gives the first index $\geq x$, `upper_bound` the first index $> x$. If they're equal, there are no copies of x. Number of copies = $hi - lo$.

Time: $O(\log n)$ per query

EXAMPLE All permutations of a string

Print every arrangement of the letters in dictionary order.

```
int main() {
    string s; cin >> s;
    sort(s.begin(), s.end());          // start from the smallest arrangement
    do {
        cout << s << " ";
    } while (next_permutation(s.begin(), s.end()));
    cout << "\n";
}
```

INPUT

```
cab
```

OUTPUT

```
abc acb bac bca cab cba
```

How it works: `next_permutation` changes the string to the next one in dictionary order and returns false after the last one. Sort first or you'll miss some.

EXAMPLE Largest number from a list (custom comparator)

Arrange the numbers so that joined together they form the largest possible number.

```
int main() {
    int n; cin >> n;
    vector<string> v(n);
    for (auto &s : v) cin >> s;
    sort(v.begin(), v.end(), [](const string& a, const string& b) {
        return a + b > b + a;          // "9"+"34" = "934" beats "349"
    });
    if (v[0] == "0") { cout << "0\n"; return 0; } // all zeros
    for (auto &s : v) cout << s;
    cout << "\n";
}
```

INPUT

```
5
3 30 34 5 9
```

OUTPUT

```
9534330
```

How it works: compare two numbers by which order of joining them gives a bigger result. Handle the all-zeros case separately.

EXAMPLE Sort by number of 1-bits, then by value

Order 0..8 by how many 1s are in their binary form; ties go by the number itself.

```
int main() {
    vector<int> v = {0, 1, 2, 3, 4, 5, 6, 7, 8};
    sort(v.begin(), v.end(), [](int a, int b) {
        int pa = __builtin_popcount(a), pb = __builtin_popcount(b);
        if (pa != pb) return pa < pb;
        return a < b;
    });
    for (int x : v) cout << x << " ";
    cout << "\n";
}
```

OUTPUT

```
0 1 2 4 8 3 5 6 7
```

EXAMPLE K-th smallest with nth_element

Find the k-th smallest number in $O(n)$ on average.

```
int main() {
    int n, k; cin >> n >> k;
    vector<int> a(n);
    for (auto &x : a) cin >> x;
    nth_element(a.begin(), a.begin() + (k - 1), a.end()); // k-th smallest -> index k-1
    cout << a[k - 1] << "\n";
}
```

INPUT

```
6 3
7 10 4 3 20 15
```

OUTPUT

```
7
```

How it works: `nth_element` only puts the right value at that one position (smaller ones before it, bigger ones after, in no order). That's faster than a full sort.

EXAMPLE Range sums with prefix sums

Answer many "sum from index l to r" queries quickly.

```
int main() {
    int n, q; cin >> n >> q;
    vector<long long> a(n), pre(n + 1, 0);
    for (auto &x : a) cin >> x;
    partial_sum(a.begin(), a.end(), pre.begin() + 1); // pre[i] = a[0] + ... + a[i-1]
    while (q--) {
        int l, r; cin >> l >> r; // 0-indexed, inclusive
        cout << pre[r + 1] - pre[l] << "\n";
    }
}
```

INPUT

```
4 2
2 4 1 3
0 3
1 2
```

OUTPUT

```
10
5
```

How it works: build the prefix array once in $O(n)$; then every query is one subtraction.

Time: $O(n)$ setup, $O(1)$ per query

EXAMPLE Maximum and where it is

Print the largest value and the index of its first occurrence, plus the minimum.

```
int main() {
    vector<int> a = {4, 9, 2, 9, 1};
    auto it = max_element(a.begin(), a.end());
    cout << "max " << *it << " at index " << (it - a.begin()) << "\n";
    cout << "min " << *min_element(a.begin(), a.end()) << "\n";
    cout << "sum " << accumulate(a.begin(), a.end(), 0LL) << "\n";
    cout << "count of 9: " << count(a.begin(), a.end(), 9) << "\n";
}
```

OUTPUT

```
max 9 at index 1
min 1
sum 25
count of 9: 2
```

How it works: these functions return iterators; subtract `a.begin()` to get the index.

12 Bits, bitset & maths helpers

```
__builtin_popcount(x);    // number of 1 bits in an int    (popcountll for long long)
__builtin_ctz(x);         // trailing zeros (x must not be 0)
__builtin_clz(x);         // leading zeros (x must not be 0)
x & 1                     // is x odd?
x & (1 << i)               // is bit i set?
x | (1 << i)               // set bit i
x & ~(1 << i)              // clear bit i
x & (x - 1)               // remove the lowest set bit; ==0 means x is a power of 2
1LL << 40                  // shifting past 31? use 1LL, or it overflows

for (int mask = 0; mask < (1 << n); mask++)    // all subsets of n items
    for (int i = 0; i < n; i++)
        if (mask & (1 << i)) { /* item i is in this subset */ }

bitset<8> b(5);            // 00000101
b.count(); b[0]; b.set(3); b.reset(0); b.to_string();

gcd(a, b); lcm(a, b);      // C++17, in <numeric>
abs(x); sqrtl(x); pow(a, b) // pow uses doubles: DON'T use it for exact integer powers
```

Rounding up division: $(a + b - 1) / b$ for positive a, b . **Modulo of a negative number:** $((x \% \text{MOD}) + \text{MOD}) \% \text{MOD}$, because in C++ $-7 \% 3$ is -1 , not 2 .

Worked examples

EXAMPLE All subsets using bitmasks

Print every subset of the items. Bit i of the mask says whether item i is in the subset.

```
int main() {
    vector<char> items = {'a', 'b', 'c'};
    int n = items.size();
    for (int mask = 0; mask < (1 << n); mask++) {
        cout << "{";
        for (int i = 0; i < n; i++)
            if (mask & (1 << i)) cout << items[i];
        cout << "} ";
    }
    cout << "\n";
}
```

OUTPUT

```
{ } {a} {b} {ab} {c} {ac} {bc} {abc}
```

How it works: there are 2^n masks from 0 to 2^n-1 , one for each subset. Use this when $n \leq 20$.

EXAMPLE The number that appears once (XOR)

Every number appears twice except one. Find it with $O(1)$ extra memory.

```
int main() {
    int n; cin >> n;
    int x = 0;
    for (int i = 0; i < n; i++) { int v; cin >> v; x ^= v; }
    cout << x << "\n";
}
```

INPUT

```
5
4 1 2 1 2
```

OUTPUT

```
4
```

How it works: $a \wedge a = 0$ and $a \wedge 0 = a$, so every pair cancels out and only the single number is left.

EXAMPLE Powers of two and bit counts

For each number print how many 1-bits it has and whether it's a power of two.

```
int main() {
    long long x;
    while (cin >> x) {
        bool pow2 = x > 0 && (x & (x - 1)) == 0;
        cout << x << ": bits=" << __builtin_popcountll(x)
              << (pow2 ? " power of 2" : "") << "\n";
    }
}
```

INPUT

```
1 6 8 12 1024
```

OUTPUT

```
1: bits=1 power of 2
6: bits=2
8: bits=1 power of 2
12: bits=2
1024: bits=1 power of 2
```

How it works: `x & (x - 1)` removes the lowest 1-bit. A power of two has only one 1-bit, so the result is 0.

EXAMPLE Subset sum with bitset

Can we pick some of the numbers so that they add up to each target?

```
int main() {
    int n; cin >> n;
    bitset<100001> can; // can[s] = 1 if sum s is reachable
    can[0] = 1;
    for (int i = 0; i < n; i++) { int x; cin >> x; can |= (can << x); }
    int q; cin >> q;
    while (q--) { int t; cin >> t; cout << t << (can[t] ? " YES" : " NO") << "\n"; }
}
```

INPUT

```
3
3 5 7
3
12 11 15
```

OUTPUT

```
12 YES
11 NO
15 YES
```

How it works: shifting the bitset by `x` adds `x` to every sum we could already make. One line does the whole knapsack, about $64\times$ faster than a normal loop.

Time: $O(n \cdot S / 64)$

13 Which container should I use?

If the problem needs...	Use
a list of items, access by index	<code>vector</code>
count how often each value appears	<code>unordered_map<int,int></code> (or <code>vector</code> if values are small)
check "have I seen this?"	<code>unordered_set</code>
values kept sorted, with insert/delete	<code>set</code> / <code>multiset</code>
"smallest value $\geq x$ " or nearest neighbour	<code>set::lower_bound</code> / <code>map::lower_bound</code>
always grab the max or min quickly	<code>priority_queue</code>
running median	two heaps (max-heap + min-heap) or <code>multiset</code>
last-in-first-out (matching brackets, next greater)	<code>stack</code>
level-by-level search (BFS)	<code>queue</code>
add/remove at both ends; sliding window max	<code>deque</code>
key $\rightarrow$ value lookups	<code>unordered_map</code> (fast) or <code>map</code> (sorted)
a pair or vector as a key	<code>map<pair<int,int>,int></code> / <code>set<vector<int>></code>
flags for many small items, bitmask speed-ups	<code>bitset</code> / <code>vector<bool></code>

14 Time complexity cheat table

Container	Access	Search	Insert	Delete	Notes
<code>vector</code>	$O(1)$	$O(n)$	$O(1)$ * end / $O(n)$ middle	$O(1)$ end / $O(n)$ middle	*amortised (average)
<code>deque</code>	$O(1)$	$O(n)$	$O(1)$ both ends	$O(1)$ both ends	
<code>stack</code> / <code>queue</code>	top/front $O(1)$	–	$O(1)$	$O(1)$	
<code>priority_queue</code>	top $O(1)$	–	$O(\log n)$	$O(\log n)$ top only	no search, no delete-any
<code>set</code> / <code>map</code>	–	$O(\log n)$	$O(\log n)$	$O(\log n)$	sorted, <code>lower_bound</code>
<code>multiset</code>	–	$O(\log n)$	$O(\log n)$	$O(\log n)$	<code>count(x)</code> is $O(\log n + k)$
<code>unordered_set</code> / <code>map</code>	–	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	$O(n)$ worst case
<code>sort</code>					$O(n \log n)$
<code>lower_bound</code> on <code>vector</code>		$O(\log n)$			range must be sorted

15 Ready-to-use patterns

These are the snippets that come up again and again in OAs. Each one is tested.

Prefix sums + hash map: count subarrays with sum = k

```
int subarraySumK(vector<int>& a, int k) {
    unordered_map<long long, int> seen;
    seen[0] = 1; // empty prefix
    long long sum = 0; int ans = 0;
    for (int x : a) {
        sum += x;
        if (seen.count(sum - k)) ans += seen[sum - k];
        seen[sum]++;
    }
    return ans;
}
```

Sliding window maximum (deque)

```
vector<int> slidingMax(vector<int>& a, int k) {
    deque<int> dq; // stores INDICES; their values decrease front -> back
    vector<int> res;
    for (int i = 0; i < (int)a.size(); i++) {
        while (!dq.empty() && dq.front() <= i - k) dq.pop_front(); // out of window
        while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back(); // useless now
        dq.push_back(i);
        if (i >= k - 1) res.push_back(a[dq.front()]);
    }
    return res;
}
```

Next greater element (monotonic stack)

```
vector<int> nextGreater(vector<int>& a) {
    int n = a.size();
    vector<int> ans(n, -1);
    stack<int> st; // indices still waiting for a bigger value
    for (int i = 0; i < n; i++) {
        while (!st.empty() && a[st.top()] < a[i]) { ans[st.top()] = a[i]; st.pop(); }
        st.push(i);
    }
    return ans;
}
```

Merge overlapping intervals

```
vector<vector<int>> mergeIntervals(vector<vector<int>> v) {
    sort(v.begin(), v.end());           // by start time
    vector<vector<int>> res;
    for (auto &x : v) {
        if (res.empty() || res.back()[1] < x[0]) res.push_back(x);    // no overlap
        else res.back()[1] = max(res.back()[1], x[1]);                // extend
    }
    return res;
}
```

Longest increasing subsequence in $O(n \log n)$

```
int lis(vector<int>& a) {
    vector<int> tails;                // tails[i] = smallest tail of an increasing run of length i+1
    for (int x : a) {
        auto it = lower_bound(tails.begin(), tails.end(), x);    // use upper_bound for non-
decreasing
        if (it == tails.end()) tails.push_back(x);
        else *it = x;
    }
    return tails.size();
}
```

Coordinate compression (big values $\rightarrow$ 0, 1, 2 ...)

```
vector<int> s = a;
sort(s.begin(), s.end());
s.erase(unique(s.begin(), s.end()), s.end());
for (int &x : a) x = lower_bound(s.begin(), s.end(), x) - s.begin();
// {100, 5, 100, 7} -> {2, 0, 2, 1}
```

Binary search on the answer

```
// find the SMALLEST x in [lo, hi] for which ok(x) is true (ok must be false...false true...true)
long long lo = 0, hi = 1e18;
while (lo < hi) {
    long long mid = lo + (hi - lo) / 2;    // avoids overflow
    if (ok(mid)) hi = mid;
    else lo = mid + 1;
}
// answer = lo
```

BFS on a grid

```
int dr[] = {-1, 1, 0, 0}, dc[] = {0, 0, -1, 1};
vector<vector<int>> dist(n, vector<int>(m, -1));
queue<pair<int,int>> q;
q.push({sr, sc}); dist[sr][sc] = 0;
while (!q.empty()) {
    auto [r, c] = q.front(); q.pop();
    for (int d = 0; d < 4; d++) {
        int nr = r + dr[d], nc = c + dc[d];
        if (nr < 0 || nr >= n || nc < 0 || nc >= m) continue; // outside grid
        if (grid[nr][nc] == '#' || dist[nr][nc] != -1) continue; // wall or visited
        dist[nr][nc] = dist[r][c] + 1;
        q.push({nr, nc});
    }
}
```

Dijkstra (shortest paths, non-negative weights)

```
// adj[u] = list of {v, w}
vector<long long> dijkstra(int src, vector<vector<pair<int,int>>>& adj) {
    int n = adj.size();
    vector<long long> dist(n, LLONG_MAX);
    priority_queue<pair<long long,int>, vector<pair<long long,int>>, greater<>> pq;
    dist[src] = 0; pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue; // old entry, skip it
        for (auto [v, w] : adj[u])
            if (dist[u] + w < dist[v]) { dist[v] = dist[u] + w; pq.push({dist[v], v}); }
    }
    return dist;
}
```

Disjoint Set Union (union-find)

```
struct DSU {
    vector<int> p, sz;
    DSU(int n) : p(n), sz(n, 1) { iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); } // path compression
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false; // already in the same group
        if (sz[a] < sz[b]) swap(a, b); // union by size
        p[b] = a; sz[a] += sz[b];
        return true;
    }
};
```

Fast power ($a^b \bmod m$) and the prime sieve

```
long long power(long long b, long long e, long long m = MOD) {
    long long r = 1; b %= m;
    while (e > 0) {
        if (e & 1) r = r * b % m;
        b = b * b % m;
        e >>= 1;
    }
    return r;
}

// modular inverse when MOD is prime: inv(a) = power(a, MOD - 2)

vector<int> sieve(int n) { // all primes <= n in O(n log log n)
    vector<bool> isPrime(n + 1, true);
    isPrime[0] = false; if (n >= 1) isPrime[1] = false;
    for (int i = 2; 1LL * i * i <= n; i++)
        if (isPrime[i])
            for (int j = i * i; j <= n; j += i) isPrime[j] = false;
    vector<int> primes;
    for (int i = 2; i <= n; i++) if (isPrime[i]) primes.push_back(i);
    return primes;
}
```

16 Mistakes that cost marks

1. **Overflow.** Sums and products in `int`. Use `long long` and `1LL * a * b`.
2. `accumulate(..., 0)` sums in `int`. Write `0LL`.
3. `v.size() - 1` on an empty vector becomes a huge number (unsigned).
4. `mp[x]` to check a key silently inserts it. Use `count` / `find`.
5. `multiset.erase(x)` removes all copies. Use `erase(find(x))`.
6. `lower_bound(s.begin(), s.end(), x)` on a set is $O(n)$. Use `s.lower_bound(x)`.
7. Comparator with `<=` can crash `sort`. Use `<`.
8. Forgetting the `priority_queue` is a max-heap by default.
9. Calling `top()` / `front()` / `pop()` on an empty container.
10. Erasing from a container while looping over it with a range-for. Use the iterator returned by `erase`.
11. Passing vectors by value in recursion $\rightarrow$ TLE. Use `&`.
12. `endl` in big outputs $\rightarrow$ slow. Use `"\n"`.
13. `pow()` for integer powers gives rounding errors. Write a loop or fast power.
14. Not resetting global arrays/containers between test cases.
15. Negative modulo: $((x \% \text{MOD}) + \text{MOD}) \% \text{MOD}$.
16. `1 << 40` overflows. Write `1LL << 40`.
17. `unordered_map` TLE on anti-hash tests. Switch to `map` or a custom hash.
18. Deep recursion ($n \approx 10^5$ – 10^6) can overflow the stack. Use an explicit stack or BFS.

17 STL interview questions

Interviewers like to check that you know what's *inside* the tools you use. Short answers you can give confidently:

Q. map vs unordered_map?

map is a red-black tree: keys sorted, $O(\log n)$ per operation. unordered_map is a hash table: no order, $O(1)$ average but $O(n)$ worst case when many keys collide.

Q. How does vector grow?

When it's full it allocates a bigger block (usually $1.5\times$ to $2\times$ the size), copies or moves the elements across, and frees the old block. That's why push_back is $O(1)$ amortised: the occasional expensive copy is spread over many cheap pushes.

Q. size vs capacity? What does reserve do?

size = elements stored; capacity = space already allocated. reserve(n) allocates space up front so no regrowth happens while you push up to n items.

Q. vector vs list?

vector is one continuous block: fast indexing, cache-friendly. list is a doubly linked list: $O(1)$ insert/erase at a known position, but no indexing and poor cache use. In practice vector is usually faster.

Q. What algorithm does std::sort use?

Introsort: quicksort, switching to heapsort if recursion gets too deep, and insertion sort for small parts. Guaranteed $O(n \log n)$. It's not stable; use stable_sort if order of equal items matters.

Q. What is iterator invalidation?

After some changes, old iterators/pointers no longer point to valid elements. Example: push_back that causes a vector to regrow invalidates all its iterators. Erasing from a map invalidates only the erased element's iterator.

Q. How is priority_queue built?

It's a binary heap stored inside a vector. push and pop are $O(\log n)$, top is $O(1)$.

Q. What is a hash collision and how is it handled?

Two keys hash to the same bucket. The standard unordered_map keeps a list per bucket (chaining) and rehashes into more buckets when the load factor gets too high.

Q. Why can't I use a pair as an unordered_map key directly?

There's no built-in std::hash for pair. Either use map (which only needs <), or write your own hash.

Q. emplace_back vs push_back?

push_back takes an already-built object; emplace_back builds the object in place from its constructor arguments, which can avoid a temporary copy.

Q. What does stable mean for a sort?

Elements that compare equal keep their original relative order.

Q. What are lower_bound and upper_bound?

On a sorted range: lower_bound gives the first element $\geq x$, upper_bound gives the first element $> x$. Both are $O(\log n)$.

18 Before-you-submit checklist

- ☐ Did I read the constraints and is my complexity fast enough?
- ☐ Could any value overflow int? (sums, products, counts)
- ☐ Did I test $n = 1$, $n = 0$ (if allowed), all elements equal?
- ☐ Did I test the largest values?
- ☐ Negative numbers handled? Negative modulo fixed?
- ☐ Containers and globals reset between test cases?
- ☐ No debug prints left in the output?
- ☐ Output format exactly right? (spaces, new lines, case)
- ☐ On a function-stub problem: no extra `main()` or `cin`?
- ☐ Stuck for 20 minutes? Submit the brute force for partial marks.

Good luck with your placements. Practise typing these until they feel automatic.